

Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en
Sistemas Computacionales

Unidad de Aprendizaje:
Inteligencia Artificial
Grupo: 6CV2

Práctica: A*

Alumno:
Barros Martínez Luis Enrique
Bautista Ríos Alfredo
Garcia Martinez Angel Yahel

Profesor: Marco Antonio Castillo

Introducción

El código presentado implementa una solución para el algoritmo de búsqueda **A***, ampliamente utilizado en el ámbito de la inteligencia artificial y la informática para encontrar el camino más corto entre dos puntos en un espacio discretizado, como un mapa o una cuadrícula. Este algoritmo combina la búsqueda de menor costo con una heurística que aproxima la distancia restante al objetivo, logrando un equilibrio entre eficiencia y precisión. Su popularidad radica en su capacidad para encontrar soluciones óptimas en problemas de enrutamiento, planificación de movimientos en robots, y navegación en videojuegos, entre otros.

El programa utiliza la biblioteca SDL para la visualización y el manejo de eventos, lo que permite representar gráficamente el progreso del algoritmo en tiempo real. Además, organiza el flujo del programa en diversas funciones y clases que facilitan su comprensión y extensibilidad. Desde el manejo de colores y texturas hasta la lógica central del algoritmo, el código ofrece una visión integral de cómo estructurar una solución basada en A* y cómo presentar visualmente el proceso de búsqueda y su resultado.

Desarrollo

Estructura del Programa

El programa consta de tres componentes principales:

1. Manejo de la visualización con SDL:

La clase `vsr::Screen` se encarga de manejar la ventana y renderizar los elementos visuales, como líneas de la cuadrícula, texturas, y símbolos de inicio, objetivo y bloqueos. Esta parte del programa proporciona una interfaz gráfica que ilustra la cuadrícula y permite al usuario interactuar con el sistema mediante clics del mouse.

2. Algoritmo A*:

La lógica del algoritmo está implementada en la clase `Pathfinder`, que gestiona una cuadrícula donde cada celda puede ser transitable o no. Esta clase encapsula métodos clave como:

- `Get_lower_open()`: Encuentra el nodo con el menor costo heurístico en la lista abierta.

- `Check_neighbours()`: Explora los vecinos de un nodo actual, actualizando los costos acumulados y la lista abierta.
- `Step()`: Ejecuta un paso del algoritmo, moviendo nodos de la lista abierta a la cerrada y calculando caminos. Estas funciones cooperan para garantizar que el algoritmo avance de manera óptima desde el punto inicial hasta el objetivo.

3. Renderizado de Resultados y Proceso de Iteración:

El bucle principal del programa combina los datos del algoritmo con funciones de visualización para mostrar el estado actual de la búsqueda. Se representan elementos como:

- Las celdas exploradas, mediante flechas que indican la dirección del movimiento.
- El camino encontrado, coloreado en verde.
- Los nodos de inicio y destino, representados por íconos específicos.

Detalles del Algoritmo

El algoritmo implementado se basa en celdas que almacenan información como:

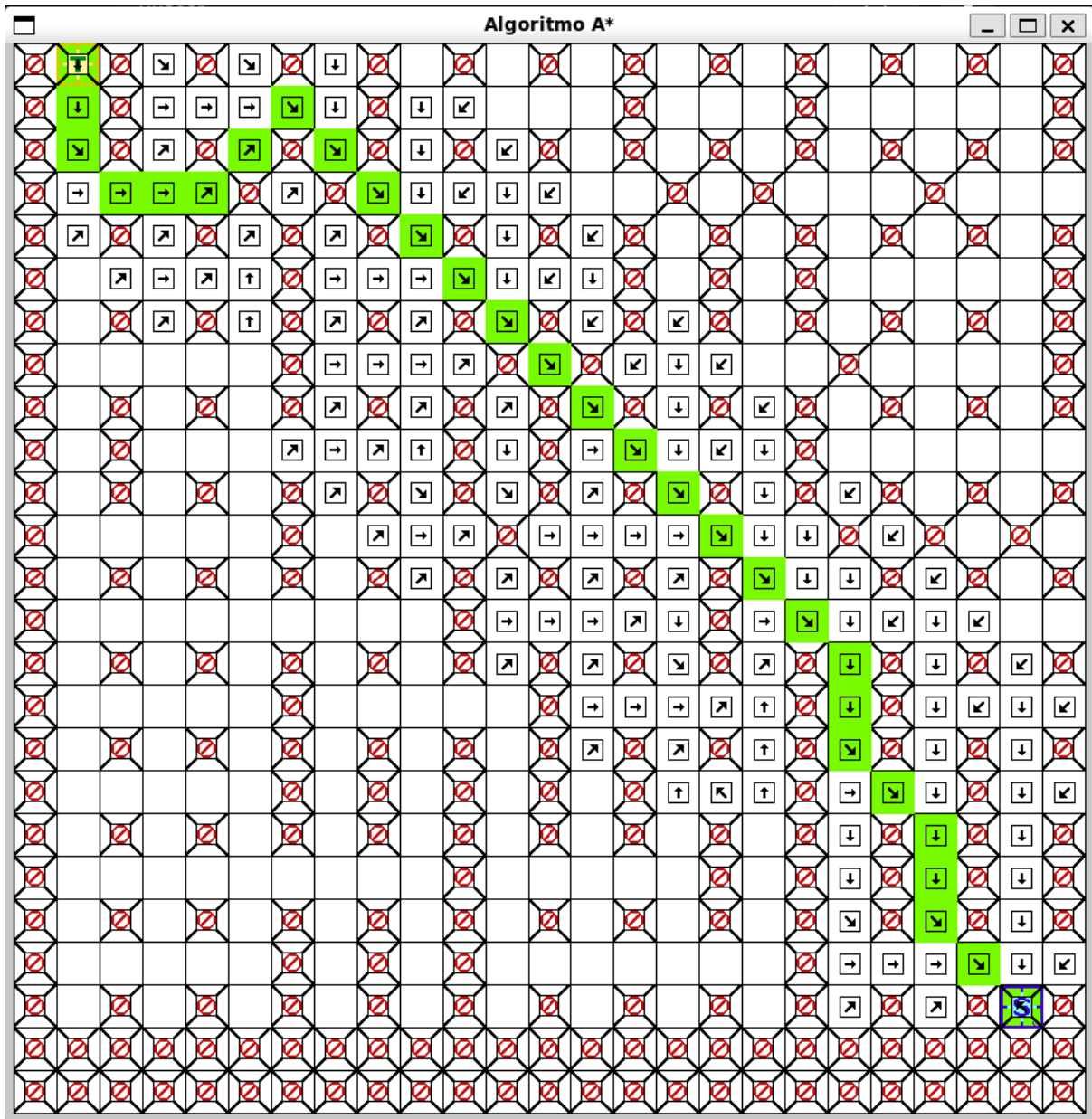
- Su posición en la cuadrícula.
- Un puntero al nodo padre para reconstruir el camino.
- Los costos acumulados (**g**), heurísticos (**h**), y totales (**f = g + h**). Cada celda es instanciada como un objeto de la clase `A_star_cell`, que proporciona métodos para actualizar su estado y determinar el índice de textura basado en la dirección del movimiento.

Gestión de Mapas y Texturas

El archivo `map.txt` contiene la representación inicial del mapa. Este se carga en `Pathfinder`, donde se genera una máscara de accesibilidad (`grid_mask`) y una representación interna para calcular caminos. Además, las texturas para flechas y otros elementos gráficos se organizan en listas, permitiendo una representación clara de los pasos del algoritmo y su resultado.

Con esta base, el programa se adapta fácilmente a diferentes tamaños de mapas, heurísticas y configuraciones visuales, haciéndolo útil para explorar y experimentar con variantes del algoritmo A*.

Ejecución



Conclusión

El código desarrollado ofrece una implementación robusta y visualmente clara del algoritmo A*, mostrando cómo estructurar un programa eficiente y modular para resolver problemas de enrutamiento en una cuadrícula. Al integrar bibliotecas como SDL para la visualización y dividir las responsabilidades en clases y funciones específicas, el programa no solo logra cumplir su objetivo funcional, sino que también sirve como un ejemplo didáctico para aprender sobre algoritmos de búsqueda y su representación gráfica.

Además, su diseño modular permite la extensibilidad, pudiendo incorporar diferentes heurísticas, mapas o métodos de visualización sin alterar significativamente la estructura base. Esto lo convierte en una herramienta versátil tanto para el aprendizaje como para aplicaciones prácticas en áreas como videojuegos, robótica y planificación de rutas. En conjunto, el proyecto destaca por su claridad, funcionalidad y potencial para futuras expansiones o adaptaciones.

Código

A_star.c++

```
#include "Pathfinder.hpp"
#include "../SDL_Visor/SDL_Visor.hpp"

const int WIDTH = 800;
const int HEIGHT = 800;

void eventos(SDL_Event& eventos);
std::map<String, vsr::Color*> Load_colors();
Vector<vsr::Icon*> Load_arrow_Icons(vsr::Screen* window);
void create_map_texture(String texture_name, uint16_t
cell_widht, Vector<Vector<bool>> map_grid, vsr::Screen* window, vsr::Icon
blocked_icon, vsr::Color &grid_color);
double heuristic(Cord target, Cord current);
Cord start, target;
uint8_t status = 0;
float escala = float(WIDTH/8);
Vector<Vector<bool>> mask;
int main(int argc, char const *argv[])
{
    //Create window
    vsr::Screen window("Algoritmo A*", WIDTH, HEIGHT, SDL_RENDERER_ACCELERATED);
    window.Set_events_handler(eventos);

    window.Init_TTF("sans", "../SDL_Visor/fonts/NotoSans/NotoSans-Thin.ttf", 10);
    window.Init_IMG();
    //Load colors
    auto colors = Load_colors();
    //Load Icons
    auto arrow_icons = Load_arrow_Icons(&window);
    vsr::Icon block_icon("../Iconos/BLOCK.png", window.Get_renderer());
    vsr::Icon start_icon("../Iconos/START.png", window.Get_renderer());
    vsr::Icon target_icon("../Iconos/TARGET.png", window.Get_renderer());
    //Load map_mask
    auto grid = Pathfinder("map.txt", heuristic);
    mask = grid.Get_grid_mask();
    escala = escala / mask.size();
    //Create background texture
```

```

uint16_t cell_size = 32;
create_map_texture("background", cell_size, grid.Get_grid_mask(),
&window, block_icon,*colors["Black"]);
Vector<Vector<uint8_t>> textures;
bool path_found = false;
int x,y;
uint16_t cell_width = WIDTH/mask.size();
uint16_t cell_height = HEIGHT/ mask[0].size();
SDL_Rect area;
while (window.Handle_events())
{
    window.Clean_screen(*colors["White"]);
    window.Draw_saved_texture("background");
    switch (status)
    {
        case 2:
            grid.Set_start(start);
            grid.Set_target(target);
            status++;
            break;
        case 3:
            if (!path_found)
            {
                path_found = grid.Step();
                if (path_found)
                {
                    status++;
                }
            }
            area =
{start.x*cell_width,start.y*cell_height,cell_width,cell_height};
            window.Draw_texture(start_icon.Get_texture(),&area);
            area =
{target.x*cell_width,target.y*cell_height,cell_width,cell_height};
            window.Draw_texture(target_icon.Get_texture(),&area);
            textures = grid.Get_textures();
            x = 0;
            for (auto &&column : textures)
            {
                y = 0;
                for (auto &&cell : column)
                {
                    if (cell != 255)
                    {
                        area =
{x*cell_width,y*cell_height,cell_width,cell_height};
window.Draw_texture(arrow_icons[cell]->Get_texture(),&area);
                    }

                    y++;
                }
            }
        }
    }
}

```

```

        }
        x++;
    }

    break;
default:

    Vector<Cord> path = grid.Get_path();

    for (auto &&path_cell : path){

        area = {path_cell.x
*cell_width,path_cell.y*cell_height,cell_width,cell_height};
        window.Draw_filled_rectangle(path_cell.x
*cell_width,path_cell.y*cell_height,cell_width,cell_height,*colors["Green"]);
    }
    area =
{start.x*cell_width,start.y*cell_height,cell_width,cell_height};
    window.Draw_texture(start_icon.Get_texture(),&area);
    area =
{target.x*cell_width,target.y*cell_height,cell_width,cell_height};
    window.Draw_texture(target_icon.Get_texture(),&area);

    x = 0;
    for (auto &&column : textures)
    {
        y = 0;
        for (auto &&cell : column)
        {
            if (cell != 255)
            {
                SDL_Rect area =
{x*cell_width,y*cell_height,cell_width,cell_height};
window.Draw_texture(arrow_icons[cell]->Get_texture(),&area);
            }

            y++;
        }
        x++;
    }
    break;
}

window.Present_renderer();

SDL_Delay(300);
}

return 0;

```

```

}

void eventos(SDL_Event& evento) {
switch (evento.type)
{
case SDL_MOUSEBUTTONDOWN:
    switch (evento.button.button)
    {
        case SDL_BUTTON_LEFT:
            std::cout << floor(evento.button.x/(escala*8)) << " " <<
floor(evento.button.y/(escala*8)) << std::endl;
            switch (status)
            {
                case 0:
                    start.x = evento.button.x/(escala*8);
                    start.y = evento.button.y/(escala*8);
                    if (mask[start.x][start.y])
                    {
                        status++;
                    }

                    break;
                case 1:
                    target.x = evento.button.x/(escala*8);
                    target.y = evento.button.y/(escala*8);
                    if (mask[target.x][target.y])
                    {
                        status++;
                    }
                    break;
                default:
                    break;
            }
            break;

        default:
            break;
    }
    break;

default:
    break;
}
}

std::map<String,vsr::Color*> Load_colors() {
    std::map<String, vsr::Color*> colors = {
        {"White", new vsr::Color{255, 255, 255, 255}},
        {"Black", new vsr::Color(0, 0, 0, 255)},
        {"Green", new vsr::Color(124, 252, 0, 255)},
        {"Orange",new vsr::Color(255, 128, 0, 255)},
        {"Red", new vsr::Color(237, 64, 64, 255)}
    }
}

```

```

};
return colors;
}

Vector<vsr::Icon *> Load_arrow_Icons(vsr::Screen* window)
{
    Vector<vsr::Icon *> icons;
    icons.push_back(new
vsr::Icon("../Iconos/ULA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/UA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/URA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/RA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/DRA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/DA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/DLA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/LA.png",window->Get_renderer()));
    return icons;
}

void create_map_texture(String texture_name, uint16_t cell_size,
Vector<Vector<bool>> map_grid, vsr::Screen *window, vsr::Icon blocked_icon,
vsr::Color &grid_color){
    uint16_t width = cell_size * map_grid.size();
    uint16_t height = cell_size * map_grid[0].size();
    window->Create_texture(texture_name, width+1, height+1);
    window->Clean_screen();
    // Dibujar líneas verticales
    for (uint16_t i = 0; i <= map_grid.size(); i++){
        window->Draw_line_pos(i * cell_size, 0, i * cell_size, height,
grid_color);
    }
    // Dibujar líneas horizontales
    for (uint16_t i = 0; i <= map_grid[0].size(); i++){
        window->Draw_line_pos(0, i * cell_size, width, i * cell_size,
grid_color);
    }
    for (uint16_t u = 0; u < map_grid[0].size(); u++)
    {
        for (uint16_t i = 0; i < map_grid.size(); i++)
        {
            if(!map_grid[u][i]){
                SDL_Rect block_area =
{u*cell_size,i*cell_size,cell_size,cell_size};
                window->Draw_texture(blocked_icon.Get_texture(),&block_area);
            }
        }
    }
}

```

```

    }

    window->End_texture();
}

double heuristic(Cord target, Cord current){
    return std::max(std::abs(target.x - current.x), std::abs(target.y -
current.y));
}

```

Pathfinder

```

#include "Pathfinder.hpp"

Cord Pathfinder::Get_lower_open(){
    if (open.empty()) {
        return { UINT16_MAX, UINT16_MAX }; // Invalid coordinate
    }

    // Initialize with the first element
    Cord lowest_cord = open.front();
    uint lowest_weight = grid[lowest_cord.x][lowest_cord.y]->Get_weight();

    // Find the node with the lowest weight
    for (Cord& c : open) {
        uint current_weight = grid[c.x][c.y]->Get_weight();
        if (current_weight < lowest_weight) {
            lowest_weight = current_weight;
            lowest_cord = c;
        }
    }
    return lowest_cord;
}

//void Pathfinder::Check_neighbours(Cord current){}

void Pathfinder::Check_neighbours(Cord current) {
    static const int dx[] = { -1, -1, -1, 0, 0, 1, 1, 1 };
    static const int dy[] = { -1, 0, 1, -1, 1, -1, 0, 1 };

    for (int dir = 0; dir < 8; ++dir) {
        int nx = current.x + dx[dir];
        int ny = current.y + dy[dir];

        // Boundary checks
        if (nx < 0 || ny < 0 || nx >= grid_mask.size() || ny >=
grid_mask[0].size()) {
            continue;
        }

        Cord neighbour = { static_cast<uint16_t>(nx),
static_cast<uint16_t>(ny) };

        // Skip if not passable or already closed
        if (!grid_mask[nx][ny] || std::find(closed.begin(), closed.end(),
neighbour) != closed.end()) {
            continue;
        }
    }
}

```

```

        uint tentative_g = grid[current.x][current.y]->Get_accumulated() + 1;
// Assuming uniform cost

        // If neighbor is not in open list, add it
        if (std::find(open.begin(), open.end(), neighbour) == open.end()) {
            grid[nx][ny] = new A_star_cell(current, neighbour, target,
tentative_g, heuristic_function);
            grid[nx][ny]->Set_parent(current);
            open.push_back(neighbour);
        } else {
            // Update if a better path is found
            if (tentative_g < grid[nx][ny]->Get_accumulated()) {
                grid[nx][ny]->Update(tentative_g, tentative_g +
grid[nx][ny]->Get_heuristic(), current);
            }
        }
    }
}

uint16_t Pathfinder::A_star_cell::last_id = 0;
Pathfinder::Pathfinder(String map_file_path)
{
    start_counter = false;
    path_found = false;
    std::ifstream infile(map_file_path);
    if (!infile.is_open()) {
        std::cerr << "Error: Can't open file: " << map_file_path <<
std::endl;
        return;
    }

    String line;
    // Obtener dimensiones
    if (std::getline(infile, line)) {
        std::istringstream iss(line);
        int rows, cols;
        if (!(iss >> rows >> cols)) {
            std::cerr << "Error: Wrong format for the header in file: " <<
map_file_path << std::endl;
            infile.close();
            return;
        }

        grid_mask.resize(cols, Vector<bool>(rows, false));
        grid.resize(cols, Vector<A_star_cell*>(rows, nullptr));

        // Load Map
        int current_row = 0;
        while (std::getline(infile, line) && current_row < rows) {
            if (line.length() < static_cast<size_t>(cols)) {
                std::cerr << "Warning: line " << current_row + 2 << " is
incomplete. Filling with '#' " << std::endl;
            }
            for (int col = 0; col < cols; ++col) {
                if (col < static_cast<int>(line.length())) {
                    char ch = line[col];
                    if (ch != '#') {
                        grid_mask[col][current_row] = true;
                    }
                } else {

```

```

        grid_mask[col][current_row] = false;
    }
}
current_row++;
}
// Validate size
if (current_row < rows) {
    std::cerr << "Warning: Map dimensions are wrong. Filling with
'#'." << std::endl;
}
} else {
    std::cerr << "Error: void file: " << map_file_path << std::endl;
}
infile.close();
start_counter = 0;
}

Pathfinder::Pathfinder(String map_file_path, double (*heuristic) (Cord target,
Cord current)):Pathfinder(map_file_path) {
    heuristic_function = heuristic;
}

Vector<Vector<bool>> Pathfinder::Get_grid_mask() {
    return grid_mask;
}

void Pathfinder::Set_start(Cord start) {
    this->start = start;
    if (start_counter < 2) {
        start_counter++;
    }
}

void Pathfinder::Set_target(Cord end) {
    this->target = end;
    if (start_counter < 2) {
        start_counter++;
    }
}

bool Pathfinder::Step() {
//std::cout << "DF 1" <<std::endl;
    if (start_counter < 2) {
        std::cerr << "Error: Can't start, missing start or target" <<
std::endl;
        return false;
    }
//std::cout << "DF 2" <<std::endl;
    // Initialize the start node if the open list is empty
    if (open.empty()) {
//std::cout << "DF 2.0" <<std::endl;
        grid[start.x][start.y] = new A_star_cell(start, start, target, 0,
heuristic_function);
//std::cout << "DF 2.1" <<std::endl;
        open.push_back(start);
    }
//std::cout << "DF 3" <<std::endl;
    // No path exists if the open list is empty after initialization
    if (open.empty()) {
        std::cerr << "Error: No path found." << std::endl;
        return false;
    }
}

```

```

    }
    //std::cout << "DF 4" <<std::endl;
    // Get the node with the lowest weight (f = g + h) from the open list
    Cord current = Get_lower_open();
    //std::cout << "DF 5" <<std::endl;
    // Check if the target is reached
    // std::cout << current.x << " " << current.y << "target " << target.x <<
    " " << target.y<< std::endl;

    if (current.x == target.x && current.y == target.y) {
        //std::cout << "Path found!" << std::endl;
        path_found = true;
        // Optional: Reconstruct path here by backtracking using parent nodes
        return true;
    }
    //std::cout << "DF 6" <<std::endl;
    // Remove current node from the open list
    //std::cout << "Removed from open " << current.x << " " << current.y <<
    std::endl;
    open.erase(std::remove(open.begin(), open.end(), current), open.end());
    //std::cout << "DF 7" <<std::endl;
    // Add current node to the closed list
    closed.push_back(current);
    //std::cout << "DF 8" <<std::endl;
    // Explore neighbors
    static const int dx[] = { -1, -1, -1, 0, 0, 1, 1, 1 };
    static const int dy[] = { -1, 0, 1, -1, 1, -1, 0, 1 };
    //std::cout << "DF 9" <<std::endl;
    for (int dir = 0; dir < 8; ++dir) {
        int nx = current.x + dx[dir];
        int ny = current.y + dy[dir];
    //std::cout << "DF 10" <<std::endl;
        // Check boundaries
        if (nx < 0 || ny < 0 || nx >= grid_mask.size() || ny >=
grid_mask[0].size()) {
            continue;
        }
    //std::cout << "DF 11" <<std::endl;
        Cord neighbour = { static_cast<uint16_t>(nx),
static_cast<uint16_t>(ny) };

        // Skip if the cell is not passable or already in the closed list
        if (!grid_mask[nx][ny] || std::find(closed.begin(), closed.end(),
neighbour) != closed.end()) {
            continue;
        }
    //std::cout << "DF 12" <<std::endl;
        double tentative_g = grid[current.x][current.y]->Get_accumulated() +
Distance(current,neighbour);
    //std::cout << "DF 13" <<std::endl;
        // If the neighbor is not in the open list, create a new A_star_cell
        if (std::find(open.begin(), open.end(), neighbour) == open.end()) {
            grid[nx][ny] = new A_star_cell(current, neighbour, target,
tentative_g, heuristic_function);
            open.push_back(neighbour);
        } else {
            // If the neighbor is already in the open list, check if the new
path is better
    //std::cout << "Tentative" << tentative_g << " Current: " <<
grid[nx][ny]->Get_accumulated() << std::endl;
            if (tentative_g <= grid[nx][ny]->Get_accumulated()) {

```

```

        grid[nx][ny]->Update(tentative_g,tentative_g +
grid[nx][ny]->Get_heuristic(),current);
    }
}
}
return false;
}

Vector<Vector<uint8_t>> Pathfinder::Get_textures() {
    // Initialize a 2D vector of uint8_t with the same size as the grid
    Vector<Vector<uint8_t>> textures(grid.size(),
Vector<uint8_t>(grid[0].size(), 0));

    // Iterate through the grid to populate the textures
    for (size_t x = 0; x < grid.size(); ++x) {
        for (size_t y = 0; y < grid[x].size(); ++y) {
            if (grid[x][y] != nullptr) {
                // Assign the texture value of the corresponding A_star_cell
                textures[x][y] = grid[x][y]->Get_texture();
            } else {
                // Assign a default value (e.g., 0 for uninitialized cells)
                textures[x][y] = 255;
            }
        }
    }

    return textures;
}

Vector<Cord> Pathfinder::Get_path()
{
    auto current = target;
    Vector<Cord>path;
    if (path_found)
    {
        while (current != start)
        {
            path.push_back(current);
            current = grid[current.x][current.y]->Get_parent();
        }
        path.push_back(current);
    }
    return path;
}

Pathfinder::~Pathfinder() {
}

uint8_t Pathfinder::A_star_cell::GetTextureIndex(int dx, int dy)
{
    if (dx == -1 && dy == -1) return 0; // UP LEFT ARROW
    if (dx == 0 && dy == -1) return 1; // UP ARROW
    if (dx == 1 && dy == -1) return 2; // UP RIGHT ARROW
    if (dx == 1 && dy == 0) return 3; // RIGHT ARROW
    if (dx == 1 && dy == 1) return 4; // DOWN RIGHT ARROW
    if (dx == 0 && dy == 1) return 5; // DOWN ARROW
    if (dx == -1 && dy == 1) return 6; // DOWN LEFT ARROW
    if (dx == -1 && dy == 0) return 7; // LEFT ARROW
    return 0; // Default case;
}

```

```

}

Pathfinder::A_star_cell::A_star_cell(Cord parent, Cord position, Cord target,
uint accumulated, double (*heuristic)(Cord target, Cord current))
{
    if (heuristic == nullptr)
    {
        std::cerr << "Error: heuristic is nullptr" << std::endl;
    }

    //std::cout << "DF A1" <<std::endl;
    this->heuristic = heuristic(target, position);
    //std::cout << "DF A1.1" <<std::endl;
    this->accumulated = accumulated;
    this->parent = parent;
    this->position = position;
    //std::cout << "DF A2" <<std::endl;
    weight = accumulated + this->heuristic;

    ID = last_id;
    last_id++;

    // Calculate direction from current node to parent node
    int dx = parent.x - position.x;
    int dy = parent.y - position.y;
    //std::cout << "DF A3" <<std::endl;
    // Map direction to texture index
    texture = GetTextureIndex(dx, dy);
    //std::cout << "DF A4" <<std::endl;
}

void Pathfinder::A_star_cell::Set_parent(Cord new_parent){
    parent = new_parent;
}

void Pathfinder::A_star_cell::Update(uint new_accumulated, uint new_weigth,
Cord new_parent){
    accumulated = new_accumulated;
    weight = new_weigth;
    parent = new_parent;
    int dx = parent.x - position.x;
    int dy = parent.y - position.y;
    texture = GetTextureIndex(dx, dy);
}

double Pathfinder::A_star_cell::Get_accumulated()
{
    return accumulated;
}

double Pathfinder::A_star_cell::Get_heuristic()
{
    return heuristic;
}

double Pathfinder::A_star_cell::Get_weight()
{
    return weight;
}

Cord Pathfinder::A_star_cell::Get_parent()

```

```
{
    return Cord(parent);
}

uint8_t Pathfinder::A_star_cell::Get_texture()
{
    return texture;
}

Pathfinder::A_star_cell::~A_star_cell()
{
}

double Distance(const Cord& point1, const Cord& point2) {
    // Use the distance formula: sqrt((x2 - x1)^2 + (y2 - y1)^2)
    return std::sqrt(std::pow(point2.x - point1.x, 2) + std::pow(point2.y -
point1.y, 2));
}
```



Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en
Sistemas Computacionales



Unidad de Aprendizaje:

Inteligencia Artificial

Grupo: 6CV2

(DFS y BFS)

Integrantes:

Barros Martínez Luis Enrique
Bautista Ríos Alfredo
Garcia Martinez Angel Yahel

Profesor: Marco Antonio Castillo

Introducción

La presente práctica aborda nuestro acercamiento a los algoritmos de búsqueda no informados (DFS y BFS), estos consisten en la selección e implementación de estrategias de búsqueda de un estado solución a partir de un estado inicial sin introducir al algoritmo de solución conocimiento sobre el impacto de las transiciones en la exploración del espacio de estados.

Tras ser implementados dichos algoritmos se obtendrá su análisis respecto al tiempo que les toma llegar a el dato en búsqueda con respecto a los casos ingresados.

Dicho de la siguiente manera los programas elevaorados tiene que tener graficos, ademas de estar implementado bajo un grid que simula un laberito

Desarrollo

Se realizará la programación del algoritmo de Búsqueda en anchura (BFS) y Búsqueda en profundidad (DFS) en esta ocasión en lenguaje C++

BFS

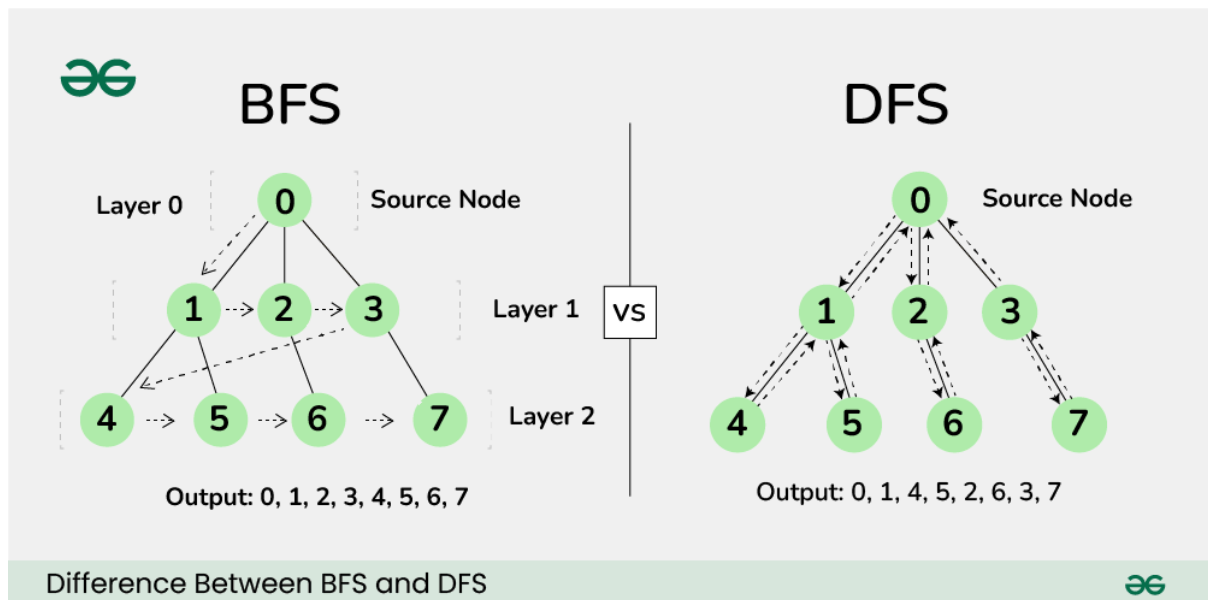
La búsqueda en anchura provee un algoritmo simple (basado en el recorrido de árboles por niveles) para encontrar un estado de solución desde un estado inicial. Al implementarse mediante el recorrido por niveles, se requiere una estructura cola (FIFO, first-in, first-out).

En inteligencia artificial, el algoritmo Breadth-First Search (BFS) es una herramienta esencial para explorar y navegar por diversos espacios de problemas. Al recorrer sistemáticamente estructuras de grafos o árboles, BFS resuelve tareas como la búsqueda de rutas, el enrutamiento de redes y la resolución de acertijos. Este artículo analiza los conceptos básicos de BFS, sus algoritmos y aplicaciones prácticas en IA.

DFS

Una búsqueda en profundidad (DFS) es un algoritmo de búsqueda para lo cual recorre los nodos de un grafo. Su funcionamiento consiste en ir expandiendo cada uno de los nodos que va localizando, de forma recurrente (desde el nodo padre hacia el nodo hijo). Cuando ya no quedan más nodos que visitar en dicho camino, regresa al nodo predecesor, de modo que repite el mismo proceso con cada uno de los vecinos del nodo. Cabe resaltar que si se encuentra el nodo antes de recorrer todos los nodos, concluye la búsqueda.

La búsqueda en profundidad surge como un algoritmo crucial en inteligencia artificial que ofrece tanto conocimientos teóricos como aplicaciones prácticas. Los DFS son una herramienta fundamental para atravesar espacios de estados complejos. A medida que la tecnología evoluciona, DFS sigue siendo un algoritmo fundamental para impulsar avances y ampliar los límites de la investigación y las aplicaciones de la IA.



Código:

El código mostrado a continuación es capaz de recibir cualquier grafo luego de indicarle (número de nodos, número de aristas, y las conexiones), se genera un número aleatorio dentro del rango del número de nodos, el cual será el nodo a encontrar realizando búsquedas con los algoritmos DFS y BFS, primero mostrar el proceso que lleva a cabo la bfs con un cierto delay, terminando este proceso mostrara el camino por medio de una DFS que no siempre tiene que ser igual

```
#include <SFML/Graphics.hpp>
#include <queue>
#include <vector>
#include <iostream>
#include <utility>
#include <cmath>
#include <climits>
using namespace std;
// Laberinto
vector<vector<int>> maze = {
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    {0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1},
    {0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0},
    {1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1},
    {1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0},
    {0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1},
    {1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1},
    {0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0}
};

const int CELL_SIZE = 40; // Tamaño de cada celda en píxeles

//DFS
vector<sf::Vector2i> directions = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
void
dfs(vector<int>current,vector<vector<int>>&visited,vector<vector<sf::Vector2
i>>&prev,int &n,int &m){
    if(visited[current[0]][current[1]])return;
    visited[current[0]][current[1]]=1;
    for(auto dir:directions){
        int nx = current[0] + dir.x; // Nueva posición en x
        int ny = current[1] + dir.y; // Nueva posición en y
        if (nx >= 0 && ny >= 0 && nx < n && ny < m && maze[nx][ny] == 0){
            if(!visited[nx][ny]){
                prev[nx][ny]={current[0],current[1]};
                dfs({nx,ny},visited,prev,n,m);
            }
        }
    }
}
```

```

}
vector<sf::Vector2i> dfs_aux(sf::Vector2i start, sf::Vector2i end) {
    int rows = maze.size(); // Número de filas
    int cols = maze[0].size(); // Número de columnas

    // Matriz de visitados
    vector<vector<int>> visited(rows, vector<int>(cols,0));
    // Matriz para guardar el nodo previo en el camino más corto
    vector<vector<sf::Vector2i>> prev(rows, vector<sf::Vector2i>(cols, {-1,
-1}));
    // Ejecutar dfs
    dfs({start.x,start.y},visited,prev,rows,cols);
    // Reconstrucción del camino desde el nodo final al inicio
    vector<sf::Vector2i> path;
    for (sf::Vector2i at = end; at != sf::Vector2i{-1, -1}; at =
prev[at.x][at.y]) {
        path.push_back(at);
    }
    reverse(path.begin(), path.end()); // Se invierte el camino para ir de
inicio a fin
    return path;
}
// Algoritmo BFS
vector<sf::Vector2i> bfs(sf::Vector2i start, sf::Vector2i end) {
    int rows = maze.size(); // Número de filas
    int cols = maze[0].size(); // Número de columnas

    // Matriz de visitados
    vector<vector<int>> visited(rows, vector<int>(cols,0));
    // Matriz para guardar el nodo previo en el camino más corto
    vector<vector<sf::Vector2i>> prev(rows, vector<sf::Vector2i>(cols, {-1,
-1}));
    // Cola para explorar los nodos
    queue<vector<int>>q;
    // Inicializar el nodo de inicio
    q.push({start.x, start.y});
    // Direcciones posibles para moverse (derecha, abajo, izquierda, arriba)
    vector<sf::Vector2i> directions = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
    prev[start.x][start.y]={-1,-1};
    visited[start.x][start.y]=1;
    while(q.size()){
        auto current=q.front();
        q.pop();
        for(auto dir:directions){
            int nx = current[0] + dir.x; // Nueva posición en x
            int ny = current[1] + dir.y; // Nueva posición en y
            if (nx >= 0 && ny >= 0 && nx < rows && ny < cols && maze[nx][ny]
== 0){

```

```

        if(!visited[nx][ny]){
            q.push({nx,ny});
            visited[nx][ny]=1;
            prev[nx][ny]={current[0],current[1]};
        }
    }
}

// Reconstrucción del camino desde el nodo final al inicio
vector<sf::Vector2i> path;
for (sf::Vector2i at = end; at != sf::Vector2i{-1, -1}; at =
prev[at.x][at.y]) {
    path.push_back(at);
}
cout<<endl;
reverse(path.begin(), path.end()); // Se invierte el camino para ir de
inicio a fin
return path;
}

int main() {
    int rows = maze.size(); // Número de filas
    int cols = maze[0].size(); // Número de columnas

    // Crear ventana
    sf::RenderWindow window(sf::VideoMode(cols * CELL_SIZE, rows *
CELL_SIZE), "Dijkstra Maze Solver");

    sf::Vector2i start{-1, -1}, end{-1, -1}; // Coordenadas del inicio y fin
    int selecting = 0; // Bandera 0 inicio 1 fin

    // Bucle principal de la aplicación
    while (window.isOpen()) {
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed)
                window.close(); // Cerrar ventana

            // Detección de clics del mouse para inicio y fin
            if (event.type == sf::Event::MouseButtonPressed) {
                int x = event.mouseButton.y / CELL_SIZE; // Coordenada x en
celdas
                int y = event.mouseButton.x / CELL_SIZE; // Coordenada y en
celdas

                if (x >= 0 && y >= 0 && x < rows && y < cols && maze[x][y] ==
0) {
                    if (selecting==0) {

```

```

        start = {x, y}; // Asignar inicio
        end={-1,-1};
        selecting^=1; // Pasar a seleccionar el fin
    } else {
        end = {x, y}; // Asignar fin
        selecting^=1; // Pasar a seleccionar inicio, Si
continua
    }
}
}

window.clear(sf::Color::White); // Limpiar la ventana

// Dibujar el laberinto
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        sf::RectangleShape cell(sf::Vector2f(CELL_SIZE - 1, CELL_SIZE
- 1));

        cell.setPosition(j * CELL_SIZE, i * CELL_SIZE);

        // Color de las celdas
        if (maze[i][j] == 1)
            cell.setFillColor(sf::Color::Black); // Muros o barreras
        else
            cell.setFillColor(sf::Color::White); // Espacio
transitable

        // Colorear inicio y fin
        if (start == sf::Vector2i(i, j))
            cell.setFillColor(sf::Color::Green); // Inicio
        if (end == sf::Vector2i(i, j))
            cell.setFillColor(sf::Color::Red); // Fin

        // Dibujar borde negro a cada celda
        cell.setOutlineThickness(1);
        cell.setOutlineColor(sf::Color::Black);

        window.draw(cell); // Dibujar celda
    }
}

// Dibujar el camino más corto si el inicio y fin están definidos
if (start != sf::Vector2i{-1, -1} && end != sf::Vector2i{-1, -1}) {
    auto path = bfs(start, end);
    window.display();
    sf::sleep(sf::seconds(2));
    for (auto &pos : path) {

```

```

        sf::ConvexShape triangulo;
        triangulo.setPointCount(3);
        triangulo.setPoint(0, sf::Vector2f(pos.y * CELL_SIZE, pos.x *
CELL_SIZE)); // Vértice superior
        triangulo.setPoint(1, sf::Vector2f(pos.y *
CELL_SIZE+CELL_SIZE-1, pos.x * CELL_SIZE)); // Vértice inferior izquierdo
        triangulo.setPoint(2, sf::Vector2f(pos.y * CELL_SIZE, pos.x *
CELL_SIZE+CELL_SIZE-1)); // Vértice inferior derecho
        triangulo.setFillColor(sf::Color::Cyan); // Color del camino
        triangulo.setOutlineThickness(1);
        triangulo.setOutlineColor(sf::Color::Black);
        window.draw(triangulo); // Dibujar celda del camino
        window.display();
        sf::sleep(sf::seconds(0.5));
    }
    path=dfs_aux(start,end);
    for (auto &pos : path) {
        sf::ConvexShape triangulo;
        triangulo.setPointCount(3);
        triangulo.setPoint(0, sf::Vector2f(pos.y * CELL_SIZE, pos.x *
CELL_SIZE+CELL_SIZE-1)); // Vértice superior
        triangulo.setPoint(1, sf::Vector2f(pos.y *
CELL_SIZE+CELL_SIZE-1, pos.x * CELL_SIZE+CELL_SIZE-1)); // Vértice inferior
izquierdo
        triangulo.setPoint(2, sf::Vector2f(pos.y *
CELL_SIZE+CELL_SIZE-1, pos.x * CELL_SIZE)); // Vértice inferior derecho
        sf::RectangleShape cell(sf::Vector2f(CELL_SIZE - 1, CELL_SIZE
- 1));

        triangulo.setFillColor(sf::Color::Yellow); // Color del
camino

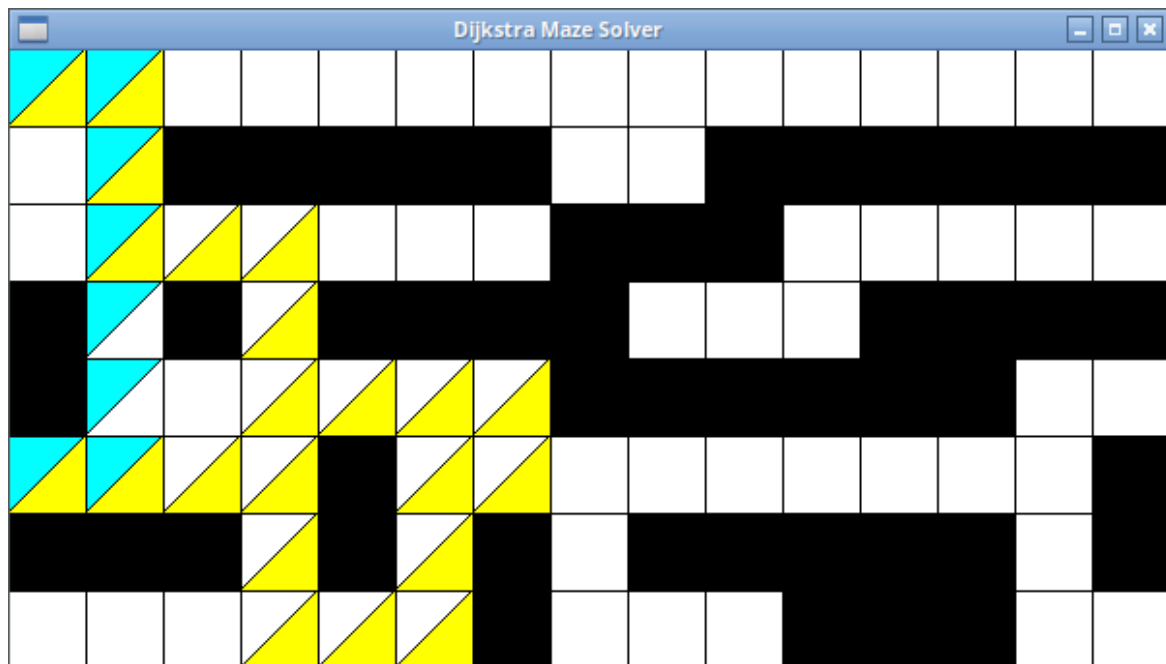
        triangulo.setOutlineThickness(1);
        triangulo.setOutlineColor(sf::Color::Black);
        window.draw(triangulo); // Dibujar celda del camino
        window.display();
        sf::sleep(sf::seconds(0.5));
    }
    window.display();
    sf::sleep(sf::seconds(2));
}

window.display(); // Mostrar la ventana
}

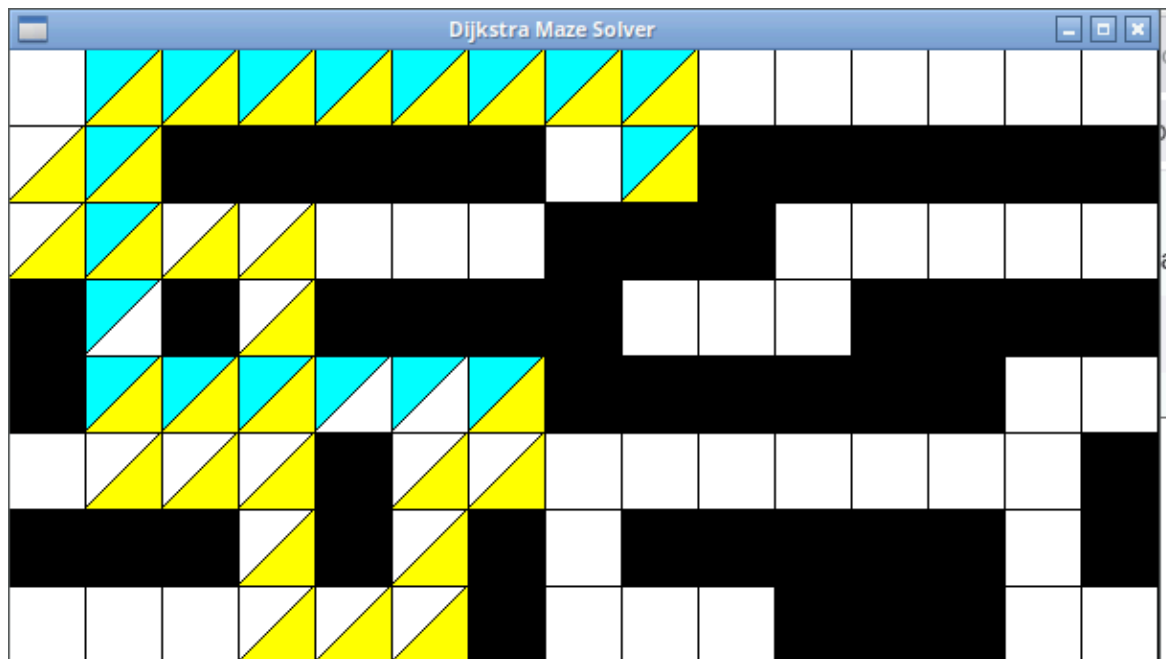
return 0;
}

```

Ejemplo Caso 1:



Ejemplo Caso 2:



Conclusión

La conclusion mas importante de la practica es que la bfs nos ayuda a encontrar un nodo con el menor numero de pasos , razon por la cual la bfs se utiliza en algoritmos de caminos cortos como el dijkstra, pero eso no hace menos importante a los algoritmos como la BFS ya que es un algoritmo mas rapido de implementar que una bfs, pero tiene una desventaja la pila recursiva, para un numero mayor de datos esto podria ser un problema pero no demasiado grande, con lo cual siempre podemos recurrir a un BFS. En conlcusion se puedenm usar ambos para casi la mayoria de datos, unos tienen ciertas propiedades y otros son mas sencillos de implementar.

Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en
Sistemas Computacionales

Unidad de Aprendizaje:
Inteligencia Artificial
Grupo: 6CV2

Práctica: Dijkstra

Alumno:
Barros Martínez Luis Enrique
Bautista Ríos Alfredo
Garcia Martinez Angel Yahel

Profesor: Marco Antonio Castillo

Introducción

El programa presentado utiliza la biblioteca SFML (Simple and Fast Multimedia Library) para implementar una solución visual y funcional del algoritmo de Dijkstra. Este algoritmo es ampliamente conocido por calcular el camino más corto en un grafo ponderado, y aquí se aplica a una representación de laberinto en una cuadrícula. La visualización permite a los usuarios seleccionar puntos de inicio y fin en un laberinto interactivo y observar cómo el algoritmo determina el camino óptimo, resaltando este camino en pantalla.

La integración de SFML para renderizar el laberinto y el camino más corto hace que este programa sea no solo funcional, sino también visualmente intuitivo. Este diseño facilita tanto el aprendizaje del algoritmo de Dijkstra como su aplicación en problemas prácticos como navegación, planificación de rutas y juegos, al permitir una interacción directa con el entorno.

Desarrollo

Estructura del Programa

El programa se divide en tres componentes principales: el manejo del laberinto, la implementación del algoritmo de Dijkstra y la visualización interactiva. Cada una de estas partes juega un rol esencial en la funcionalidad y claridad del programa:

1. **Representación del Laberinto:** El laberinto está definido como una matriz bidimensional de enteros (`maze`), donde:
 - `0` representa espacios transitables.
 - `1` representa obstáculos. Esta representación simplifica la lógica del algoritmo y permite la fácil adaptación a diferentes configuraciones de laberintos.
2. **Algoritmo de Dijkstra:** El algoritmo implementado se encarga de calcular el camino más corto desde un punto de inicio hasta un punto de destino en el laberinto. Se utiliza una cola de prioridad (`std::priority_queue`) para explorar los nodos de menor costo acumulado primero. Las principales características del algoritmo incluyen:
 - Una matriz de costos mínimos (`dist`) para rastrear el costo más bajo a cada nodo.
 - Una matriz de nodos previos (`prev`) para reconstruir el camino una vez alcanzado el destino.

- Exploración de vecinos en las cuatro direcciones cardinales, verificando límites y transitabilidad.
- 3. **Interfaz Gráfica:** La biblioteca SFML maneja la visualización y la interacción del usuario. Algunas características clave son:
 - **Selección del inicio y fin:** Los usuarios pueden hacer clic en la cuadrícula para definir el punto de partida (verde) y el destino (rojo).
 - **Dibujado del laberinto:** Cada celda se representa como un rectángulo, con diferentes colores para obstáculos, camino y nodos especiales (inicio/fin).
 - **Camino más corto:** Una vez calculado, el camino se dibuja en celdas de color cian, resaltando el resultado del algoritmo.
 -

Implementación del Algoritmo

El algoritmo de Dijkstra está encapsulado en la función `dijkstra`, que toma como entrada las posiciones de inicio y fin. Este realiza lo siguiente:

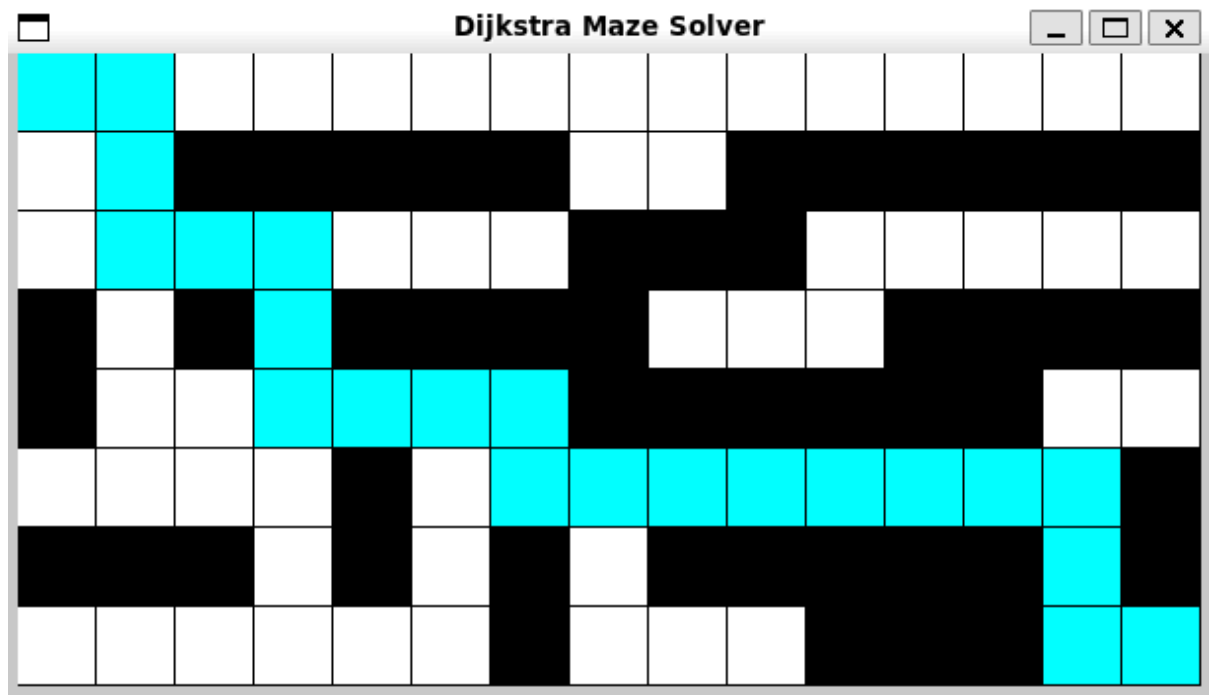
- Inicializa los costos de todas las celdas a infinito (`INT_MAX`) excepto el nodo de inicio, cuyo costo es 0.
- Utiliza una cola de prioridad para explorar los nodos en orden ascendente de costo acumulado.
- Actualiza los costos y caminos previos al encontrar rutas más cortas hacia nodos vecinos.
- Reconstruye el camino final desde el destino hacia el inicio utilizando la matriz `prev`.
-

Gestión de Gráficos

El uso de SFML permite representar de manera clara el estado del laberinto y el progreso del algoritmo:

- Las celdas se dibujan con un tamaño configurable (`CELL_SIZE`) y colores diferenciados para obstáculos, espacios libres, inicio, fin y el camino encontrado.
- El programa actualiza dinámicamente la ventana para reflejar la interacción del usuario y los resultados del algoritmo.
- Los clics del mouse permiten definir puntos de inicio y fin, lo que habilita la recalculación automática del camino.

Ejecución



Conclusión

Este programa ofrece una implementación funcional y visualmente atractiva del algoritmo de Dijkstra, utilizando la biblioteca SFML para crear una interfaz gráfica interactiva que facilita su comprensión y experimentación. La representación del laberinto como una cuadrícula simple y el uso de colores diferenciados para cada tipo de celda hacen que el proceso de búsqueda sea intuitivo y fácil de seguir. Además, la posibilidad de seleccionar dinámicamente los puntos de inicio y fin fomenta la exploración y prueba de diferentes escenarios.

El diseño modular y claro del código permite adaptaciones y mejoras, como la integración de diferentes algoritmos de búsqueda o la expansión a laberintos más complejos. En conjunto, el programa no solo es una herramienta útil para aprender sobre Dijkstra, sino que también tiene potencial para aplicaciones prácticas en áreas como videojuegos, navegación y resolución de problemas de rutas en grafos. Su combinación de eficiencia, interactividad y visualización lo convierte en un ejemplo destacado de cómo llevar conceptos algorítmicos a implementaciones prácticas y accesibles.

Código

```
#include <SFML/Graphics.hpp>
#include <queue>
#include <vector>
#include <iostream>
#include <utility>
#include <cmath>
#include <climits>
using namespace std;

// Laberinto definido como una matriz de enteros (0: espacio libre, 1: obstáculo)
vector<vector<int>> maze = {
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    {0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1},
    {0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0},
    {1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1},
    {1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0},
    {0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1},
    {1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1},
    {0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0}
};

const int CELL_SIZE = 40; // Tamaño de cada celda en píxeles

// Estructura para representar un nodo del laberinto
struct Node {
    int x, y, cost; // Coordenadas y costo acumulado para llegar a este nodo
    bool operator>(const Node &other) const { return cost > other.cost; } //
// Comparador para la cola de prioridad
};

// Implementación del algoritmo de Dijkstra para encontrar el camino más corto
vector<sf::Vector2i> dijkstra(sf::Vector2i start, sf::Vector2i end) {
    int rows = maze.size(); // Número de filas
    int cols = maze[0].size(); // Número de columnas

    // Matriz de costos mínimos desde el nodo inicial
    vector<vector<int>> dist(rows, vector<int>(cols, INT_MAX));
    // Matriz para guardar el nodo previo en el camino más corto
    vector<vector<sf::Vector2i>> prev(rows, vector<sf::Vector2i>(cols, {-1, -1}));
    // Cola de prioridad para explorar los nodos
    priority_queue<Node, vector<Node>, greater<Node>> pq;

    // Inicializar el nodo de inicio
    pq.push({start.x, start.y, 0});
    dist[start.x][start.y] = 0;

    // Direcciones posibles para moverse (derecha, abajo, izquierda, arriba)
    vector<sf::Vector2i> directions = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    // Bucle principal del algoritmo
    while (!pq.empty()) {
        Node current = pq.top();
        pq.pop();

        // Si llegamos al nodo final, terminamos
        if (sf::Vector2i(current.x, current.y) == end)
            break;

        // Explorar los vecinos
        for (auto dir : directions) {
            int nx = current.x + dir.x; // Nueva posición en x
            int ny = current.y + dir.y; // Nueva posición en y

            // Verificar si el vecino está dentro de los límites y es transitable
```

```

        if (nx >= 0 && ny >= 0 && nx < rows && ny < cols && maze[nx][ny] == 0)
        {
            int new_cost = current.cost + 1; // Costo acumulado
            // Si encontramos un camino más corto, actualizamos
            if (new_cost < dist[nx][ny]) {
                dist[nx][ny] = new_cost;
                prev[nx][ny] = {current.x, current.y};
                pq.push({nx, ny, new_cost});
            }
        }
    }

    // Reconstrucción del camino desde el nodo final al inicio
    vector<sf::Vector2i> path;
    for (sf::Vector2i at = end; at != sf::Vector2i{-1, -1}; at = prev[at.x][at.y])
    {
        path.push_back(at);
    }
    reverse(path.begin(), path.end()); // Invertimos el camino para que vaya del
    inicio al final
    return path;
}

int main() {
    int rows = maze.size(); // Número de filas
    int cols = maze[0].size(); // Número de columnas

    // Crear la ventana de renderizado
    sf::RenderWindow window(sf::VideoMode(cols * CELL_SIZE, rows * CELL_SIZE),
    "Dijkstra Maze Solver");

    sf::Vector2i start{-1, -1}, end{-1, -1}; // Coordenadas del inicio y fin
    bool selectingStart = true; // Bandera para determinar si estamos seleccionando
    el inicio

    // Bucle principal de la aplicación
    while (window.isOpen()) {
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed)
                window.close(); // Cerrar ventana

            // Detectar clics del mouse para seleccionar inicio y fin
            if (event.type == sf::Event::MouseButtonPressed) {
                int x = event.mouseButton.y / CELL_SIZE; // Coordenada x en celdas
                int y = event.mouseButton.x / CELL_SIZE; // Coordenada y en celdas

                if (x >= 0 && y >= 0 && x < rows && y < cols && maze[x][y] == 0) {
                    if (selectingStart) {
                        start = {x, y}; // Asignar inicio
                        selectingStart = false; // Pasar a seleccionar el fin
                    } else {
                        end = {x, y}; // Asignar fin
                    }
                }
            }
        }

        window.clear(sf::Color::White); // Limpiar la ventana

        // Dibujar el laberinto
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                sf::RectangleShape cell(sf::Vector2f(CELL_SIZE - 1, CELL_SIZE -
1));
                cell.setPosition(j * CELL_SIZE, i * CELL_SIZE);

```

```

        // Color de las celdas
        if (maze[i][j] == 1)
            cell.setFillColor(sf::Color::Black); // Obstáculo
        else
            cell.setFillColor(sf::Color::White); // Espacio libre

        // Colorear inicio y fin
        if (start == sf::Vector2i(i, j))
            cell.setFillColor(sf::Color::Green); // Inicio
        if (end == sf::Vector2i(i, j))
            cell.setFillColor(sf::Color::Red); // Fin

        // Dibujar borde negro
        cell.setOutlineThickness(1);
        cell.setOutlineColor(sf::Color::Black);

        window.draw(cell); // Dibujar celda
    }
}

// Dibujar el camino más corto si el inicio y fin están definidos
if (start != sf::Vector2i{-1, -1} && end != sf::Vector2i{-1, -1}) {
    auto path = dijkstra(start, end);
    for (auto &pos : path) {
        sf::RectangleShape cell(sf::Vector2f(CELL_SIZE - 1, CELL_SIZE -
1));

        cell.setPosition(pos.y * CELL_SIZE, pos.x * CELL_SIZE);
        cell.setFillColor(sf::Color::Cyan); // Color del camino
        cell.setOutlineThickness(1);
        cell.setOutlineColor(sf::Color::Black);
        window.draw(cell); // Dibujar celda del camino
    }
}

window.display(); // Mostrar la ventana
}

return 0;
}

```

Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en
Sistemas Computacionales

Unidad de Aprendizaje:
Inteligencia Artificial
Grupo: 6CV2

Práctica: Menú de algoritmos
de búsqueda

Alumno:
Barros Martínez Luis Enrique
Bautista Ríos Alfredo
Garcia Martinez Angel Yahel

Profesor: Marco Antonio Castillo

Introducción

El programa presentado es una implementación visual e interactiva de varios algoritmos de búsqueda en grafos, utilizando **SDL2** como biblioteca gráfica. Permite a los usuarios experimentar con diferentes estrategias de búsqueda, como BFS, DFS, Dijkstra y A*, en un laberinto representado como una cuadrícula. Los usuarios pueden seleccionar un punto de inicio y un objetivo, y el programa calcula y visualiza el camino más corto entre ellos según el algoritmo elegido.

El diseño combina funcionalidad y facilidad de uso, proporcionando botones para elegir el algoritmo deseado y un botón de reinicio para realizar múltiples experimentos. Además, el programa muestra visualmente el progreso y el resultado de cada algoritmo, destacando su comportamiento y diferencias en tiempo de ejecución y eficiencia en la búsqueda de rutas.

Desarrollo

Estructura del Programa

El programa se divide en varias secciones principales, cada una enfocada en un aspecto específico:

1. Interfaz Gráfica:

La interfaz gráfica se implementa utilizando **SDL2** para renderizar el laberinto y los elementos interactivos. Algunos elementos clave son:

- **Botones:** Se encuentran en una columna lateral y permiten seleccionar los algoritmos BFS, DFS, A*, y Dijkstra, además de un botón de reinicio.
- **Laberinto:** Representado como una cuadrícula donde las celdas son obstáculos (#) o espacios libres (_). Las celdas se colorean según su rol (inicio, objetivo, camino, o espacio libre).

2. Lógica de Búsqueda:

La clase **Pathfinding** encapsula la implementación de los algoritmos:

- **BFS (Breadth-First Search):** Explora los nodos por nivel, garantizando encontrar el camino más corto en grafos no ponderados.
- **DFS (Depth-First Search):** Explora nodos profundamente antes de retroceder, siendo menos eficiente para encontrar caminos cortos pero útil en ciertas aplicaciones.

- **A (A Star):*** Combina la exploración por costo acumulado y una heurística (distancia Manhattan) para encontrar caminos óptimos de manera eficiente.
- **Dijkstra:** Variante que utiliza la distancia mínima acumulada, idéntica a A* cuando se omite la heurística.

3. Interactividad del Usuario:

Los usuarios pueden seleccionar un punto de inicio y un objetivo haciendo clic en el laberinto. Los botones permiten alternar entre algoritmos y reiniciar la simulación.

Detalles de Implementación

1. Representación del Laberinto:

El laberinto es una matriz bidimensional de caracteres (**maze**), donde cada celda indica si es un obstáculo (**#**) o un espacio libre (**_**). Esto simplifica la validación de movimientos en los algoritmos.

2. Algoritmos de Búsqueda:

- **BFS y DFS:** Implementados utilizando estructuras estándar (**std::queue** y **std::stack**) para explorar nodos en diferentes órdenes. Ambos reconstruyen el camino al objetivo utilizando una estructura **cameFrom** que rastrea el nodo anterior de cada celda visitada.
- **A y Dijkstra:*** Usan una cola de prioridad para explorar nodos según el costo estimado más bajo. A* combina el costo acumulado con una heurística, mientras que Dijkstra utiliza solo el costo acumulado.
- **Reutilización de Código:** La implementación de Dijkstra reutiliza A* al ignorar la heurística.

3. Visualización del Camino:

Durante la ejecución de un algoritmo, el camino se dibuja progresivamente con un retardo (**SDL_Delay**) para ilustrar el proceso. Las celdas en el camino se colorean de azul, mientras que el inicio y el objetivo permanecen destacados en verde y rojo, respectivamente.

4. Interfaz Gráfica:

La función **drawMaze** es responsable de renderizar todo el laberinto, los botones y los caminos generados. También gestiona los colores según el estado de cada celda.

Flujo del Programa

1. El programa comienza con la inicialización de SDL2 y la configuración de la ventana.
2. Los usuarios seleccionan el inicio y el objetivo en el laberinto, y luego eligen un algoritmo mediante los botones.
3. El algoritmo seleccionado calcula el camino más corto entre los puntos y lo dibuja progresivamente.
4. Los usuarios pueden reiniciar la simulación para probar con diferentes configuraciones.

Conclusión

Este programa combina la visualización interactiva con la funcionalidad de varios algoritmos de búsqueda en grafos, proporcionando una herramienta educativa y experimental. Su estructura modular facilita la adición de nuevos algoritmos o ajustes en el laberinto. La visualización progresiva del camino ayuda a entender el comportamiento de cada algoritmo, destacando sus diferencias en eficiencia y exploración.

El uso de SDL2 para la interfaz gráfica asegura una representación clara y atractiva del laberinto y los caminos generados. En conjunto, este programa es una excelente plataforma para aprender y explorar técnicas de búsqueda en grafos, con aplicaciones prácticas en navegación, planificación de rutas y simulaciones.

Código

```
#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h>
#include <iostream>
#include <algorithm>
#include <queue>
#include <stack>
#include <vector>
#include <cmath>
#include <map>

const int WINDOW_WIDTH = 800;
```

```
const int WINDOW_HEIGHT = 600;
const int BUTTON_WIDTH = 200;
const int GRID_SIZE = 30;
const int CELL_SIZE = 20;
// Colores
const SDL_Color WHITE = {255, 255, 255, 255};
const SDL_Color BLACK = {0, 0, 0, 255};
const SDL_Color GREEN = {0, 255, 0, 255};
const SDL_Color RED = {255, 0, 0, 255};
const SDL_Color BLUE = {0, 0, 255, 255};
const SDL_Color GRAY = {200, 200, 200, 255};
// Estados
enum State { SELECT_START, SELECT_GOAL, RUNNING };
// Laberinto
char maze[GRID_SIZE][GRID_SIZE] = {
```

```
};

// Función para dibujar texto (requiere SDL_ttf)
void drawText(SDL_Renderer* renderer, const char* text, int x, int y, SDL_Color color, int fontSize) {
    TTF_Font* font = TTF_OpenFont("../SDL_Visor/fonts/monospace/Monospace.ttf",
fontSize); // Asegúrate de que la ruta al archivo de la fuente sea correcta
    if (font == nullptr) {
        std::cerr << "Failed to load font: " << TTF_GetError() << std::endl;
        return;
    }
    SDL_Surface* surface = TTF_RenderText_Solid(font, text, color);
    if (surface == nullptr) {
        std::cerr << "Failed to create text surface: " << TTF_GetError() <<
std::endl;
        TTF_CloseFont(font);
        return;
    }
    SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
    SDL_FreeSurface(surface);
    if (texture == nullptr) {
        std::cerr << "Failed to create text texture: " << SDL_GetError() <<
std::endl;
        TTF_CloseFont(font);
    }
}
```

```

        return;
    }

    SDL_Rect destRect = {x, y, surface->w, surface->h};
    SDL_RenderCopy(renderer, texture, nullptr, &destRect);
    SDL_DestroyTexture(texture);
    TTF_CloseFont(font);
}

// Clase para manejar los algoritmos de búsqueda
class Pathfinding {
public:
    static std::vector<std::pair<int, int>> BFS(const std::pair<int, int>& start,
const std::pair<int, int>& goal);

    static std::vector<std::pair<int, int>> DFS(const std::pair<int, int>& start,
const std::pair<int, int>& goal);

    static std::vector<std::pair<int, int>> Dijkstra(const std::pair<int, int>&
start, const std::pair<int, int>& goal);

    static std::vector<std::pair<int, int>> AStar(const std::pair<int, int>& start,
const std::pair<int, int>& goal);
};

// Verifica si una celda es válida
bool isValidCell(int x, int y);

// Dibuja el laberinto
void drawMaze(SDL_Renderer* renderer, const std::pair<int, int>& start, const
std::pair<int, int>& goal, const std::vector<std::pair<int, int>>& path);

// Programa principal
int main(int argc, char* argv[]) {
    SDL_Init(SDL_INIT_VIDEO);

    TTF_Init();

    SDL_Window* window = SDL_CreateWindow("Pathfinding Visualization",
SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED, WINDOW_WIDTH, WINDOW_HEIGHT,
SDL_WINDOW_SHOWN);

    SDL_Renderer* renderer = SDL_CreateRenderer(window, -1,
SDL_RENDERER_ACCELERATED);

    bool running = true;
    State state = SELECT_START;

    std::pair<int, int> start = {-1, -1};
    std::pair<int, int> goal = {-1, -1};
    std::vector<std::pair<int, int>> path;

    char selectedAlgorithm = 0;
    while (running) {
        SDL_Event event;

        while (SDL_PollEvent(&event)) {
            if (event.type == SDL_QUIT) {
                running = false;
            } else if (event.type == SDL_MOUSEBUTTONDOWN) {
                int x = event.button.x;

```

```

        int y = event.button.y;
        if (x < BUTTON_WIDTH) { // Botones
            if (y < 100) selectedAlgorithm = 'B'; // BFS
            else if (y < 200) selectedAlgorithm = 'D'; // DFS
            else if (y < 300) selectedAlgorithm = 'A'; // A*
            else if (y < 400) selectedAlgorithm = 'J'; // Dijkstra
            else if (y >= 400 && y < 500) { // Botón de reset
                start = {-1, -1};
                goal = {-1, -1};
                path.clear();
                state = SELECT_START;
            }

            if (selectedAlgorithm && start.first != -1 && goal.first != -1)
            {
                switch (selectedAlgorithm) {
                    case 'B': path = Pathfinding::BFS(start, goal); break;
                    case 'D': path = Pathfinding::DFS(start, goal); break;
                    case 'A': path = Pathfinding::AStar(start, goal);
break;

                    case 'J': path = Pathfinding::Dijkstra(start, goal);
break;

                }

                state = RUNNING;
            }
        } else { // Laberinto
            int gridX = (x - BUTTON_WIDTH) / CELL_SIZE;
            int gridY = y / CELL_SIZE;
            if (state == SELECT_START) {
                start = {gridX, gridY};
                state = SELECT_GOAL;
            } else if (state == SELECT_GOAL) {
                goal = {gridX, gridY};
            }
        }
    }

    SDL_SetRenderDrawColor(renderer, WHITE.r, WHITE.g, WHITE.b, WHITE.a);
    SDL_RenderClear(renderer);

    // Dibujar botones
    SDL_SetRenderDrawColor(renderer, GRAY.r, GRAY.g, GRAY.b, GRAY.a);
    SDL_Rect buttonRect = {0, 0, BUTTON_WIDTH, 100};
    for (int i = 0; i < 4; ++i) {
        SDL_RenderFillRect(renderer, &buttonRect);
    }
}

```

```

        buttonRect.y += 100;
    }
    // Dibujar el botón de reset
    SDL_Rect resetButtonRect = {0, 400, BUTTON_WIDTH, 100};
    SDL_RenderFillRect(renderer, &resetButtonRect);
    drawText(renderer, "Reset", 50, 430, BLACK, 24);
    drawText(renderer, "BFS", 50, 30, BLACK, 24);
    drawText(renderer, "DFS", 50, 130, BLACK, 24);
    drawText(renderer, "A*", 50, 230, BLACK, 24);
    drawText(renderer, "Dijkstra", 50, 330, BLACK, 24);
    // Dibujar laberinto
    drawMaze(renderer, start, goal, path);
    SDL_RenderPresent(renderer);
}

TTF_Quit();
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
return 0;
}

bool isValidCell(int x, int y) {
    return x >= 0 && x < GRID_SIZE && y >= 0 && y < GRID_SIZE && maze[y][x] == '_';
}

void drawMaze(SDL_Renderer* renderer, const std::pair<int, int>& start, const
std::pair<int, int>& goal, const std::vector<std::pair<int, int>>& path) {
    for (int y = 0; y < GRID_SIZE; ++y) {
        for (int x = 0; x < GRID_SIZE; ++x) {
            SDL_Rect cell = {BUTTON_WIDTH + x * CELL_SIZE, y * CELL_SIZE,
CELL_SIZE, CELL_SIZE};
            SDL_SetRenderDrawColor(renderer, WHITE.r, WHITE.g, WHITE.b, WHITE.a);
            if (maze[y][x] == '#') SDL_SetRenderDrawColor(renderer, BLACK.r,
BLACK.g, BLACK.b, BLACK.a);
            SDL_RenderFillRect(renderer, &cell);
            SDL_SetRenderDrawColor(renderer, GRAY.r, GRAY.g, GRAY.b, GRAY.a);
            SDL_RenderDrawRect(renderer, &cell);
        }
    }

    // Dibujar inicio y objetivo
    if (start.first != -1) {
        SDL_Rect cell = {BUTTON_WIDTH + start.first * CELL_SIZE, start.second *
CELL_SIZE, CELL_SIZE, CELL_SIZE};
        SDL_SetRenderDrawColor(renderer, GREEN.r, GREEN.g, GREEN.b, GREEN.a);
    }

```

```

        SDL_RenderFillRect(rendererer, &cell);
    }

    if (goal.first != -1) {
        SDL_Rect cell = {BUTTON_WIDTH + goal.first * CELL_SIZE, goal.second *
CELL_SIZE, CELL_SIZE, CELL_SIZE};

        SDL_SetRenderDrawColor(rendererer, RED.r, RED.g, RED.b, GREEN.a);
        SDL_RenderFillRect(rendererer, &cell);
    }

    // Dibujar camino con un delay sin parpadeos
    for (size_t i = 0; i < path.size(); ++i) {
        // Limpiar la pantalla
        //SDL_SetRenderDrawColor(rendererer, 0, 0, 0, 255); // Fondo negro
        //SDL_RenderClear(rendererer);

        // Redibujar el entorno completo
        // Redibujar el inicio
        if (start.first != -1) {
            SDL_Rect startCell = {BUTTON_WIDTH + start.first * CELL_SIZE,
start.second * CELL_SIZE, CELL_SIZE, CELL_SIZE};

            SDL_SetRenderDrawColor(rendererer, GREEN.r, GREEN.g, GREEN.b, GREEN.a);
            SDL_RenderFillRect(rendererer, &startCell);
        }

        // Redibujar el objetivo
        if (goal.first != -1) {
            SDL_Rect goalCell = {BUTTON_WIDTH + goal.first * CELL_SIZE, goal.second
* CELL_SIZE, CELL_SIZE, CELL_SIZE};

            SDL_SetRenderDrawColor(rendererer, RED.r, RED.g, RED.b, RED.a);
            SDL_RenderFillRect(rendererer, &goalCell);
        }

        // Dibujar el camino actual hasta el punto i
        SDL_SetRenderDrawColor(rendererer, BLUE.r, BLUE.g, BLUE.b, BLUE.a);
        for (size_t j = 0; j <= i; ++j) {
            SDL_Rect cell = {BUTTON_WIDTH + path[j].first * CELL_SIZE,
path[j].second * CELL_SIZE, CELL_SIZE, CELL_SIZE};

            SDL_RenderFillRect(rendererer, &cell);
        }

        // Mostrar la pantalla actualizada
        SDL_RenderPresent(rendererer);

        // Añadir el delay
    }

```

```

        SDL_Delay(200);
    }

}

std::vector<std::pair<int, int>> Pathfinding::BFS(const std::pair<int, int>& start,
const std::pair<int, int>& goal) {
    std::queue<std::pair<int, int>> q;
    std::map<std::pair<int, int>, std::pair<int, int>> cameFrom;
    q.push(start);
    cameFrom[start] = {-1, -1};
    const std::vector<std::pair<int, int>> directions = {{0, 1}, {1, 0}, {0, -1},
{-1, 0}};

    while (!q.empty()) {
        auto current = q.front();
        q.pop();
        if (current == goal) {
            // Reconstruir el camino
            std::vector<std::pair<int, int>> path;
            for (auto at = goal; at != std::pair<int, int>{-1, -1}; at =
cameFrom[at]) {
                path.push_back(at);
            }
            std::reverse(path.begin(), path.end());
            return path;
        }
        for (const auto& dir : directions) {
            std::pair<int, int> neighbor = {current.first + dir.first,
current.second + dir.second};
            if (isValidCell(neighbor.first, neighbor.second) &&
cameFrom.find(neighbor) == cameFrom.end()) {
                q.push(neighbor);
                cameFrom[neighbor] = current;
            }
        }
    }

    return {}; // No se encontró un camino
}

std::vector<std::pair<int, int>> Pathfinding::DFS(const std::pair<int, int>& start,
const std::pair<int, int>& goal) {
    std::stack<std::pair<int, int>> s;
    std::map<std::pair<int, int>, std::pair<int, int>> cameFrom;
    s.push(start);
    cameFrom[start] = {-1, -1};

```

```

    const std::vector<std::pair<int, int>> directions = {{0, 1}, {1, 0}, {0, -1},
{-1, 0}};

    while (!s.empty()) {
        auto current = s.top();
        s.pop();
        if (current == goal) {
            std::vector<std::pair<int, int>> path;
            for (auto at = goal; at != std::pair<int, int>{-1, -1}; at =
cameFrom[at]) {
                path.push_back(at);
            }
            std::reverse(path.begin(), path.end());
            return path;
        }

        for (const auto& dir : directions) {
            std::pair<int, int> neighbor = {current.first
+ dir.first, current.second + dir.second};

            if (isValidCell(neighbor.first,
neighbor.second) && cameFrom.find(neighbor) ==
cameFrom.end()) {
                s.push(neighbor);
                cameFrom[neighbor] = current;
            }
        }
    }

    return {};
}

std::vector<std::pair<int, int>> Pathfinding::AStar(const
std::pair<int, int>& start, const std::pair<int, int>&
goal) {
    auto heuristic = [](const std::pair<int, int>& a,
const std::pair<int, int>& b) {
        return abs(a.first - b.first) + abs(a.second -
b.second);
    };

```

```

};

std::priority_queue<std::pair<int, std::pair<int,
int>>>,

std::vector<std::pair<int,
std::pair<int, int>>>>,

std::greater<>> pq;

std::map<std::pair<int, int>, int> gScore;

std::map<std::pair<int, int>, std::pair<int, int>>
cameFrom;

gScore[start] = 0;

pq.push({heuristic(start, goal), start});

cameFrom[start] = {-1, -1};

const std::vector<std::pair<int, int>> directions =
{{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

while (!pq.empty()) {
    auto current = pq.top().second;
    pq.pop();

    if (current == goal) {
        std::vector<std::pair<int, int>> path;

        for (auto at = goal; at != std::pair<int,
int>{-1, -1}; at = cameFrom[at]) {
            path.push_back(at);
        }

        std::reverse(path.begin(), path.end());
        return path;
    }

    for (const auto& dir : directions) {
        std::pair<int, int> neighbor = {current.first
+ dir.first, current.second + dir.second};
    }
}

```

```

        if (isValidCell(neighbor.first,
neighbor.second)) {
            int tentativeGScore = gScore[current] + 1;
            if (gScore.find(neighbor) == gScore.end()
|| tentativeGScore < gScore[neighbor]) {
                gScore[neighbor] = tentativeGScore;
                pq.push({tentativeGScore +
heuristic(neighbor, goal), neighbor});
                cameFrom[neighbor] = current;
            }
        }
    }
}

return {};
}

std::vector<std::pair<int, int>>
Pathfinding::Dijkstra(const std::pair<int, int>& start,
const std::pair<int, int>& goal) {
    return AStar(start, goal);
}

```

Implementacion Comparativo:



Implementación Menu

